



**"RANKED 45
IN INDIA**

#University Category



**WORLD
UNIVERSITY
RANKINGS**

NO.1 PVT. UNIVERSITY IN
ACADEMIC REPUTATION IN INDIA



ACCREDITED GRADE 'A'
BY NAAC



PERFECT SCORE OF **150/150** AS A TESTAMENT
TO EXCEPTIONAL E-LEARNING METHODS

1



One rate
oooo

Adagrad
oooooooo

RMSprop
oooo

Adadelat
oooooo

Adam
oooooooo

Practice
oooooooo

Summary
oo

Applied Machine Learning

Unit 03 – Lecture 04: Adaptive Optimisers

Tofik Ali

School of Computer Science, UPES Dehradun

Autumn 2026

Adagrad, RMSprop, Adadelat, Adam — and numerical practice

Topic focus: **what if every parameter needs its own learning rate?**

Repository: <https://github.com/tali7c/Applied-Machine-Learning>

Sources: Zhang, Lipton, Li and Smola, *Dive into Deep Learning*, Ch. 12 (Optimization Algorithms).; Deisenroth, Faisal and Ong, *Mathematics for Machine Learning*, Ch. 7.; Kingma and Ba, "Adam: A Method for Stochastic Optimization," ICLR 2015.; Ruder, "An overview of gradient descent optimization algorithms," arXiv:1609.04747, 2016.



One rate
oooo

Adagrad
oooooooo

RMSprop
oooo

Adadelata
oooooo

Adam
oooooooooo

Practice
oooooooooo

Summary
oo

Quick Links

[One rate is not enough](#)

[Adagrad](#)

[RMSprop](#)

[Adadelata](#)

[Adam](#)

[Practice](#)

[Summary](#)

Agenda

One learning rate is not enough

Adagrad

RMSprop

Adadelata

Adam

Numerical practice

Summary

How this lecture works

Every idea appears twice

1. **By hand** — four optimisers worked on the same little bowl $f(w) = (w - 3)^2$, so you can compare them line by line.
2. **In code** — the same arithmetic, so you see the library does exactly that and nothing magic.

Commit before you look

Five slides ask you to predict an answer before it is revealed. Write your guess down. Being wrong on paper is how the idea sticks.

The centrepiece is **three steps of Adam by hand**. Everything before it exists to make those three steps obvious.

Two Datasets, One Seed

The convention, and a distinction worth keeping

- **One seed: 1.** Every mini-batch shuffle and every gradient-noise draw today is `np.random.default_rng(1)`, shown in the listing, never hidden in a helper.
- The sparse-feature problem is built with a *different* fixed number — and that is not sloppiness.

A shuffle seed and a dataset seed are not the same thing

Change the **shuffle** seed and you get another run of the same experiment. Change the **dataset** seed and you get a different experiment. Give them the same name and someone eventually “tidies up” by unifying them — and silently deletes the demonstration.

Every number in these slides reproduces exactly. Differ, and it is your code or your library version — not luck.

Where we are

The last lecture gave us

$w \leftarrow w - \eta \nabla J$, the ceiling $\eta < 2/L$,
mini-batches, and momentum
 $v \leftarrow \beta v + g$.

It also gave us the villain

On load_diabetes: largest curvature
8.05, smallest 0.017, so $\kappa = 470$.
Momentum cut 1605 iterations to 128.

Today's question

Momentum gives every coordinate the
same η and the **same** β .

What if each coordinate needs its own
step size — and the algorithm has to work
it out for itself?

Four answers, in historical order

Adagrad (2011) $\rightarrow$ RMSprop (2012) $\rightarrow$
Adadelata (2012) $\rightarrow$ Adam (2015).

What momentum does not fix

- Momentum changes the **direction** it steps in — an average of recent gradients instead of the current one.

What momentum does not fix

- Momentum changes the **direction** it steps in — an average of recent gradients instead of the current one.
- It does not change the **scale**. One η still multiplies every coordinate of that average.

What momentum does not fix

- Momentum changes the **direction** it steps in — an average of recent gradients instead of the current one.
- It does not change the **scale**. One η still multiplies every coordinate of that average.
- So the ceiling is still set by the steepest direction, and the shallow directions still crawl.

What momentum does not fix

- Momentum changes the **direction** it steps in — an average of recent gradients instead of the current one.
- It does not change the **scale**. One η still multiplies every coordinate of that average.
- So the ceiling is still set by the steepest direction, and the shallow directions still crawl.

The idea of this whole lecture, in one line

$$w_i \leftarrow w_i - \frac{\eta}{\sqrt{s_i}} g_i$$

What momentum does not fix

- Momentum changes the **direction** it steps in — an average of recent gradients instead of the current one.
- It does not change the **scale**. One η still multiplies every coordinate of that average.
- So the ceiling is still set by the steepest direction, and the shallow directions still crawl.

The idea of this whole lecture, in one line

$$w_i \leftarrow w_i - \frac{\eta}{\sqrt{s_i}} g_i$$

Divide each coordinate's step by something that measures **how big that coordinate's gradients have been**. Every optimiser today differs only in what s_i is.

By hand: two coordinates want two different step sizes

Take the ravine from last time, $f(w) = 0.01w_1^2 + 2w_2^2$. The Hessian is diagonal:

$$H = \begin{pmatrix} 0.02 & 0 \\ 0 & 4 \end{pmatrix}, \quad \kappa = \frac{4}{0.02} = 200.$$

	curvature λ_i	ideal step $1/\lambda_i$	
w_1 (shallow)	0.02	50.0	wants a huge step
w_2 (steep)	4.00	0.25	wants a tiny one

One number cannot be both

A single η must satisfy $\eta < 2/4 = 0.5$ or w_2 diverges. That is $200\times$ smaller than what w_1 wants.

The condition number is exactly the size of the mismatch.

A problem where one η genuinely cannot work

40 synthetic features, all with true weight exactly 1. Twenty are dense; twenty fire in progressively fewer rows.

```
rng = np.random.default_rng(0) # fixes THE DATA, not a run
dens = np.geomspace(0.25, 0.005, 20) # how often each rare one fires
for j in range(20):
    hit = rng.random(N) < dens[j]
    Xs[hit, 20 + j] = rng.standard_normal(hit.sum())
```

```
rows                4000
dense features, non-zero in 4000 to 4000 rows
rarest feature, non-zero in 18 rows
condition number  kappa    3.781e+02
every true weight is exactly 1.0
```

A rare feature contributes to a mini-batch gradient in maybe *one* row out of 4000. Its gradient is almost always zero.

Predict first #1

Plain SGD, $\eta = 0.01$, ten epochs, mini-batch $B = 32$. Every true weight is 1.0, and the training MSE does come down to 0.085.

What weight does it recover for the twenty dense features? And for the five rarest ones?

Predict first #1

Plain SGD, $\eta = 0.01$, ten epochs, mini-batch $B = 32$. Every true weight is 1.0, and the training MSE does come down to 0.085.

What weight does it recover for the twenty dense features? And for the five rarest ones?

```
SGD, eta=0.01, 10 epochs   MSE 0.0853
mean recovered weight, 20 dense features    1.0004
mean recovered weight, 5 rarest features    0.1562
the single rarest feature                   0.0583
```

Predict first #1

Plain SGD, $\eta = 0.01$, ten epochs, mini-batch $B = 32$. Every true weight is 1.0, and the training MSE does come down to 0.085.

What weight does it recover for the twenty dense features? And for the five rarest ones?

```
SGD, eta=0.01, 10 epochs   MSE 0.0853
mean recovered weight, 20 dense features    1.0004
mean recovered weight, 5 rarest features    0.1562
the single rarest feature                   0.0583
```

The dense features are perfect; the rare ones are not learnt

0.0583 against a true value of 1. The rarest feature fired in 18 rows out of 4000; at $\eta = 0.01$ that is nowhere near enough updates. **Raise η and the dense features blow up.**

Adagrad: a big step where the gradients are small

Keep one running total per coordinate — the sum of its squared gradients:

$$s_t = s_{t-1} + g_t \odot g_t \qquad w \leftarrow w - \frac{\eta}{\sqrt{s_t} + \varepsilon} \odot g_t$$

Read the denominator

A coordinate that has seen big gradients gets a *small* step; a coordinate that has barely fired gets a *large* one.

Two things to notice

$\odot$ is elementwise: this is p independent scalar rules.

$\varepsilon \approx 10^{-8}$ only stops a division by zero.

Duchi, Hazan and Singer (2011) — invented for exactly the sparse problem on the previous slide.

By hand: three steps of Adagrad

$$f(w) = (w - 3)^2, \quad g = 2(w - 3), \quad \eta = 0.5, \quad w_0 = 0, \quad s_0 = 0.$$

t	g_t	$s_t = s_{t-1} + g_t^2$	$\sqrt{s_t}$	step = $\eta g_t / \sqrt{s_t}$	w_t
1	-6.0000	36.0000	6.0000	-0.5000	0.5000
2	-5.0000	61.0000	7.8102	-0.3201	0.8201
3	-4.3598	80.0080	8.9447	-0.2437	1.0638

Step 1 is exactly η , whatever the gradient is

With $s_1 = g_1^2$ we get $g_1 / \sqrt{s_1} = \pm 1$, so the first step has size η whether g_1 is -6 or -6000 .

Adagrad is scale-free at the first step.

And already the steps are shrinking

$0.500 \rightarrow 0.320 \rightarrow 0.244$, while the gradient has only fallen from 6.0 to 4.4 . Remember that.

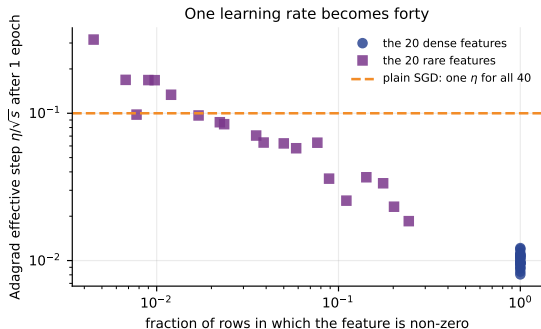
The same thing in code

```
eta, eps = 0.5, 1e-8
w, s = 0.0, 0.0
for t in (1, 2, 3):
    g = 2.0*(w - 3.0)
    s = s + g*g
    w = w - eta*g/(np.sqrt(s) + eps)
```

t=1	g= -6.0000	s= 36.0000	sqrt(s)= 6.0000	step= -0.5000	w=0.5000
t=2	g= -5.0000	s= 61.0000	sqrt(s)= 7.8102	step= -0.3201	w=0.8201
t=3	g= -4.3598	s= 80.0080	sqrt(s)= 8.9447	step= -0.2437	w=1.0638

Three lines of arithmetic. There is nothing else in Adagrad — and the printout is the hand table, digit for digit.

One learning rate becomes forty



After one epoch, $\eta = 0.10$

dense features 0.00996

rarest feature 0.3165

spread across the 40 39.3×

The whole point

You supplied *one* number. Adagrad turned it into forty, and gave the rarest feature a step 32× larger than the dense ones — because it has 32× less evidence about it.

Does it actually learn the rare features?

Same sparse problem, ten epochs, $B = 32$, each optimiser at its own best η :

sgd	eta=0.01	MSE 0.0853	dense 1.0004	5 rarest 0.1562
adagrad	eta=0.10	MSE 0.0105	dense 0.9999	5 rarest 0.8631

Does it actually learn the rare features?

Same sparse problem, ten epochs, $B = 32$, each optimiser at its own best η :

sgd	eta=0.01	MSE 0.0853	dense 1.0004	5 rarest 0.1562
adagrad	eta=0.10	MSE 0.0105	dense 0.9999	5 rarest 0.8631

0.156 $\rightarrow$ 0.863 on the rare features, and the dense ones are *still* recovered exactly. The MSE fell by a factor of eight.

Does it actually learn the rare features?

Same sparse problem, ten epochs, $B = 32$, each optimiser at its own best η :

sgd	eta=0.01	MSE 0.0853	dense 1.0004	5 rarest 0.1562
adagrad	eta=0.10	MSE 0.0105	dense 0.9999	5 rarest 0.8631

0.156 $\rightarrow$ 0.863 on the rare features, and the dense ones are *still* recovered exactly. The MSE fell by a factor of eight.

This is the case Adagrad was built for

Sparse features are the normal situation in text, in recommender systems, and in anything one-hot encoded. If your feature matrix is mostly zeros, try Adagrad before you try anything clever.

The flaw: the sum only ever grows

$s_t = \sum_{k \leq t} g_k^2$ cannot decrease — not even at the optimum, where mini-batch gradients are still non-zero.

```
st["s"] += g*g # Adagrad on load_diabetes, eta=0.05  
w = w - eta*g/(np.sqrt(st["s"]) + 1e-8)
```

update	1	s =	1.667	effective step =	0.03873
update	100	s =	11.477	effective step =	0.01476
update	1000	s =	58.171	effective step =	0.00656
update	2800	s =	148.007	effective step =	0.00411
after 200 epochs				MSE	0.4845

The flaw: the sum only ever grows

$s_t = \sum_{k \leq t} g_k^2$ cannot decrease — not even at the optimum, where mini-batch gradients are still non-zero.

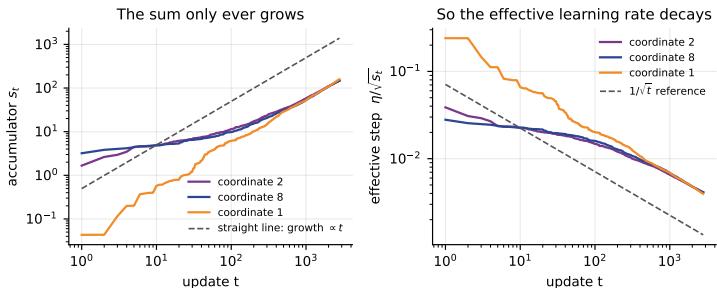
```
st["s"] += g*g # Adagrad on load_diabetes, eta=0.05  
w = w - eta*g/(np.sqrt(st["s"]) + 1e-8)
```

update	1	s =	1.667	effective step =	0.03873
update	100	s =	11.477	effective step =	0.01476
update	1000	s =	58.171	effective step =	0.00656
update	2800	s =	148.007	effective step =	0.00411
after 200 epochs				MSE	0.4845

The learning rate you set is not the one you get

By update 2800 the effective step is $9.4\times$ smaller and still falling like $1/\sqrt{t}$. Convex problems converge anyway; a deep network **stalls**.

Drawn: growth without bound



Left: three coordinates' accumulators, log-log, against a straight line of slope 1. The sum grows **linearly in t** . Right: therefore the effective step decays as $1/\sqrt{t}$ — a learning-rate schedule you never asked for and cannot switch off.

Predict first #2

A noisy one-dimensional bowl. Minimum at $w = 50$, start at $w = 0$, gradient noise of standard deviation 20 that never dies away. $\eta = 0.5$ for both.

After 20,000 updates, where has Adagrad got to? And where is RMSprop after only 100?

Predict first #2

A noisy one-dimensional bowl. Minimum at $w = 50$, start at $w = 0$, gradient noise of standard deviation 20 that never dies away. $\eta = 0.5$ for both.

After 20,000 updates, where has Adagrad got to? And where is RMSprop after only 100?

```
adagrad  t=100   w=  9.01 step=0.00055  t=1000  w= 26.50 step=0.00023
          t=5000  w= 44.80 step=0.00017  t=20000 w= 50.05 step=0.00013
rmsprop  t=100   w= 44.49 step=0.01784  t=1000  w= 50.63 step=0.02488
          t=5000  w= 53.46 step=0.02873  t=20000 w= 48.60 step=0.02334
the minimum is at w = 50
```

Predict first #2

A noisy one-dimensional bowl. Minimum at $w = 50$, start at $w = 0$, gradient noise of standard deviation 20 that never dies away. $\eta = 0.5$ for both.

After 20,000 updates, where has Adagrad got to? And where is RMSprop after only 100?

```
adagrad  t=100   w=  9.01 step=0.00055  t=1000  w= 26.50 step=0.00023
          t=5000  w= 44.80 step=0.00017  t=20000 w= 50.05 step=0.00013
rmsprop  t=100   w= 44.49 step=0.01784  t=1000  w= 50.63 step=0.02488
          t=5000  w= 53.46 step=0.02873  t=20000 w= 48.60 step=0.02334
the minimum is at w = 50
```

Adagrad needs 4,874 updates to reach RMSprop's 100th

It does arrive eventually — 50.05 is a good answer. But its step has collapsed to 1.3×10^{-4} getting there. Move the minimum now and it will never follow. Note RMSprop's own row wanders — 50.63, 53.46, 48.60 — because its step never shrank either. That is coming.

RMSprop: one symbol changes

Adagrad

$$s_t = s_{t-1} + g_t^2$$

a **sum** — unbounded

RMSprop

$$s_t = \gamma s_{t-1} + (1 - \gamma) g_t^2$$

an **average** — bounded

$$w \leftarrow w - \frac{\eta}{\sqrt{s_t} + \varepsilon} \odot g_t$$

unchanged. $\gamma = 0.9$ by default.

The same exponentially weighted average as momentum

Weight γ^j on the gradient j steps ago; total weight 1; memory about $1/(1 - \gamma) = 10$ steps. Here it averages g^2 , not g .

Hinton, Coursera lecture 6e, 2012 — famously never published as a paper.

By hand: three steps of RMSprop

Same bowl, same $\eta = 0.5$, now with $\gamma = 0.9$.

t	g_t	$s_t = 0.9s_{t-1} + 0.1g_t^2$	$\sqrt{s_t}$	step	w_t
1	-6.0000	3.6000	1.8974	-1.5811	1.5811
2	-2.8377	4.0453	2.0113	-0.7055	2.2866
3	-1.4268	3.8443	1.9607	-0.3639	2.6504

Adagrad reached 1.064; RMSprop reached 2.650

Identical η , identical bowl, target 3. Look at row 3: Adagrad's denominator was 8.94 and climbing, RMSprop's was 1.96 and it *fell* from 2.01. An average can go down.

Why the first step is 3.16η and not η : $s_1 = (1 - \gamma)g_1^2$, so $\sqrt{s_1} = \sqrt{0.1}|g_1|$ and the step is $\eta/\sqrt{1-\gamma}$.

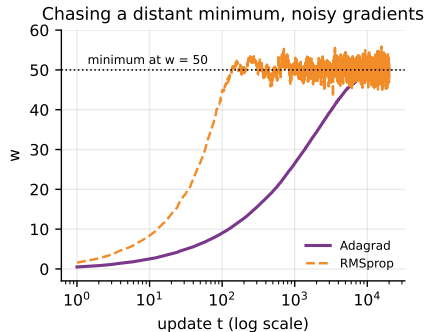
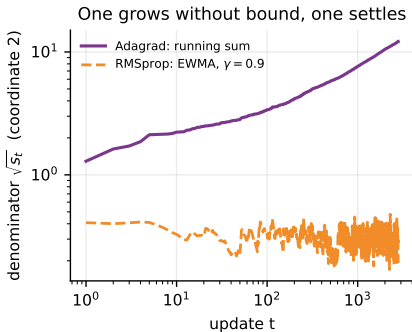
The same thing in code

```
eta, gamma = 0.5, 0.9
w, s = 0.0, 0.0
for t in (1, 2, 3):
    g = 2.0*(w - 3.0)
    s = gamma*s + (1 - gamma)*g*g # the only changed line
    w = w - eta*g/(np.sqrt(s) + 1e-8)
```

t=1	g= -6.0000	s= 3.6000	sqrt(s)= 1.8974	step= -1.5811	w=1.5811
t=2	g= -2.8377	s= 4.0453	sqrt(s)= 2.0113	step= -0.7055	w=2.2866
t=3	g= -1.4268	s= 3.8443	sqrt(s)= 1.9607	step= -0.3639	w=2.6504

Diff against the Adagrad listing: $s = s + g*g$ became $s = \text{gamma}*s + (1-\text{gamma})*g*g$.
That is the whole of RMSprop.

Two denominators, drawn



Measured, coordinate 2, 2800 updates

What RMSprop gives up

Adadelta: the units are wrong

Do a dimensional analysis of the update. Write $[w]$ for the units of a parameter and $[J]$ for the units of the loss, so g has units $[J]/[w]$.

Method	Update	Units
SGD	$-\eta g$	$[\eta] [J]/[w]$
Adagrad, RMSprop	$-\eta g / \sqrt{s}$	$[\eta] \quad (\sqrt{s} \text{ cancels } g)$
Newton	$-H^{-1}g$	$[w] \quad (\text{correct — but } H \text{ is unaffordable})$

Zeiler's observation (2012)

Adagrad and RMSprop have already cancelled the gradient's units. All that is missing is a quantity carrying the units of w — and we have one lying around: **the size of the recent updates themselves**.

$$\Delta w_t = - \frac{\text{RMS}[\Delta w]_{t-1}}{\text{RMS}[g]_t} g_t \quad \text{no } \eta \text{ anywhere}$$

Adadelta in full: two accumulators, no learning rate

gradient accumulator $s_t = \rho s_{t-1} + (1 - \rho) g_t^2$

the step $\Delta w_t = -\frac{\sqrt{d_{t-1} + \varepsilon}}{\sqrt{s_t + \varepsilon}} g_t$

update accumulator $d_t = \rho d_{t-1} + (1 - \rho) \Delta w_t^2$

parameters $w \leftarrow w + \Delta w_t$

Defaults

$\rho = 0.9$, $\varepsilon = 10^{-6}$. Note ε sits **inside** both square roots here, and it is far larger than Adagrad's 10^{-8} .

Where does the first step come from?

$d_0 = 0$, so $\text{RMS}[\Delta w]_0 = \sqrt{\varepsilon} = 10^{-3}$.
The algorithm **bootstraps itself out of** ε .

By hand and in code: Adadelta creeps, then speeds up

Same bowl $f(w) = (w - 3)^2$, $\rho = 0.9$, $\varepsilon = 10^{-6}$, $w_0 = 0$.

```
s = rho*s + (1 - rho)*g*g
```

```
dw = -np.sqrt(dacc + epsd)/np.sqrt(s + epsd) * g
```

```
dacc = rho*dacc + (1 - rho)*dw*dw
```

t	g	RMS[g]_t	RMS[dw]_{t-1}	dw	w
1	-6.0000	1.8974	0.001000	0.003162	0.003162
2	-5.9937	2.6139	0.001414	0.003243	0.006405
3	-5.9872	3.1199	0.001718	0.003297	0.009702
10	-5.9397	4.8137	0.002825	0.003485	0.033635
200	-4.3511	4.4339	0.004662	0.004575	0.829020

By hand and in code: Adadelata creeps, then speeds up

Same bowl $f(w) = (w - 3)^2$, $\rho = 0.9$, $\varepsilon = 10^{-6}$, $w_0 = 0$.

```
s = rho*s + (1 - rho)*g*g
```

```
dw = -np.sqrt(dacc + epsd)/np.sqrt(s + epsd) * g
```

```
dacc = rho*dacc + (1 - rho)*dw*dw
```

t	g	RMS[g]_t	RMS[dw]_{t-1}	dw	w
1	-6.0000	1.8974	0.001000	0.003162	0.003162
2	-5.9937	2.6139	0.001414	0.003243	0.006405
3	-5.9872	3.1199	0.001718	0.003297	0.009702
10	-5.9397	4.8137	0.002825	0.003485	0.033635
200	-4.3511	4.4339	0.004662	0.004575	0.829020

Check row 1 by hand: $\sqrt{10^{-6}} \times 6/1.8974 = 0.003162$; after 200 steps $w = 0.83$.

Adadelata buys freedom from η with a slow start.

It works out its own step size

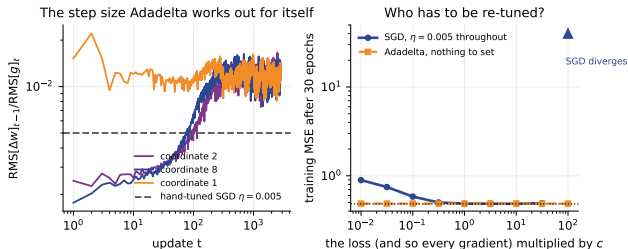
On `load_diabetes`. The ratio $r_t = \text{RMS}[\Delta w]_{t-1} / \text{RMS}[g]_t$ is a step size, so it is directly comparable with an η .

```
Adadelata's own step size  r_t = RMS[dw]_{t-1} / RMS[g]_t  (coordinate 2)
update      1      r = 0.002449
update    100      r = 0.005168
update   2800      r = 0.012292
the eta a human found by search for plain SGD  0.005000
```

It works out its own step size

On `load_diabetes`. The ratio $r_t = \text{RMS}[\Delta w]_{t-1} / \text{RMS}[g]_t$ is a step size, so it is directly comparable with an η .

```
Adadelta's own step size  r_t = RMS[dw]_{t-1} / RMS[g]_t (coordinate 2)
update    1    r = 0.002449
update   100    r = 0.005168
update  2800    r = 0.012292
the eta a human found by search for plain SGD  0.005000
```



Predict first #3

Multiply the loss by a constant c . Every gradient is multiplied by c too; nothing else changes, and *the optimum is in exactly the same place*.

**SGD keeps $\eta = 0.005$ and Adadelta keeps nothing. Who survives $c = 0.01$?
Who survives $c = 100$?**

Predict first #3

Multiply the loss by a constant c . Every gradient is multiplied by c too; nothing else changes, and *the optimum is in exactly the same place*.

**SGD keeps $\eta = 0.005$ and Adadelata keeps nothing. Who survives $c = 0.01$?
Who survives $c = 100$?**

```
multiply the loss by c; nothing else changes. 30 epochs, B=32
c          SGD eta=0.005      Adadelata
0.01              0.8941      0.4860
1                0.4860      0.4859
100              diverged    0.4859
least-squares optimum              0.4823
```

Predict first #3

Multiply the loss by a constant c . Every gradient is multiplied by c too; nothing else changes, and *the optimum is in exactly the same place*.

**SGD keeps $\eta = 0.005$ and Adadelat keeps nothing. Who survives $c = 0.01$?
Who survives $c = 100$?**

```
multiply the loss by c; nothing else changes. 30 epochs, B=32
c          SGD eta=0.005      Adadelat
0.01              0.8941      0.4860
1              0.4860      0.4859
100             diverged      0.4859
least-squares optimum              0.4823
```

Adadelat varies by a factor of 1.0002 over four decades

That is what “dimensionally consistent” buys you. A learning rate is a number in units of $[w]^2/[J]$; change the units of J and it is wrong. Adadelat’s step carries the units of w , so nothing to re-tune.

Adadelta on the real problem

load_diabetes, mini-batch $B = 32$, and **no learning rate supplied**:

Adadelta, no learning rate, 5 epochs	MSE 0.5665
Adadelta, no learning rate, 30 epochs	MSE 0.4859
least-squares optimum	MSE 0.4823

Adadelat on the real problem

load_diabetes, mini-batch $B = 32$, and **no learning rate supplied**:

Adadelat, no learning rate, 5 epochs	MSE 0.5665
Adadelat, no learning rate, 30 epochs	MSE 0.4859
least-squares optimum	MSE 0.4823

It gets there

0.4859 against an optimum of 0.4823 — as good as tuned SGD (0.4860), with nothing to tune.

It gets there last

At 5 epochs it is at 0.5665, the worst of the six optimisers we are about to compare. The slow start is real.

Adadelta on the real problem

load_diabetes, mini-batch $B = 32$, and **no learning rate supplied**:

Adadelta, no learning rate, 5 epochs	MSE 0.5665
Adadelta, no learning rate, 30 epochs	MSE 0.4859
least-squares optimum	MSE 0.4823

It gets there

0.4859 against an optimum of 0.4823 — as good as tuned SGD (0.4860), with nothing to tune.

It gets there last

At 5 epochs it is at 0.5665, the worst of the six optimisers we are about to compare. The slow start is real.

PyTorch's Adadelta still takes an `lr`, defaulting to 1.0: a pure multiplier on Δw , not a step size.

Adam: momentum + RMSprop, both as averages

first moment (direction) $m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$ from momentum

second moment (scale) $v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$ from RMSprop

bias correction $\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$

the step $w \leftarrow w - \frac{\eta \hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}$

Defaults: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$, $\eta = 0.001$. Kingma and Ba, ICLR 2015.

Note the $(1 - \beta)$ convention

Both moments are **true** running averages — the second of the two momentum conventions, not $v \leftarrow \beta v + g$. That is exactly why a bias-correction line is needed at all.

Why the first step is biased

Start with $m_0 = v_0 = 0$ and unroll one step:

$$m_1 = (1 - \beta_1) g_1 = 0.1 g_1, \quad v_1 = (1 - \beta_2) g_1^2 = 0.001 g_1^2.$$

Both are pulled towards **zero** — m_1 is one tenth of the size it should be, v_1 one thousandth.

After t steps of a constant gradient g

$$m_t = (1 - \beta_1^t) g \quad \text{so} \quad \hat{m}_t = \frac{m_t}{1 - \beta_1^t} = g. \quad \text{Unbiased, by construction.}$$

The mismatch is the problem, not the shrinkage

m shrinks by 10; $\sqrt{v}$ shrinks by $\sqrt{1000} = 31.6$. They do *not* cancel, and the ratio $m/\sqrt{v}$ comes out $31.6/10 = 3.16$ times **too large**.

By hand: Adam, step 1

$$f(w) = (w - 3)^2, \eta = 0.1, \beta_1 = 0.9, \beta_2 = 0.999, w_0 = 0, m_0 = v_0 = 0.$$

gradient	$g_1 = 2(0 - 3) = -6$
first moment	$m_1 = 0.9(0) + 0.1(-6) = -0.6$
second moment	$v_1 = 0.999(0) + 0.001(36) = 0.036$
correct m	$\hat{m}_1 = -0.6 / (1 - 0.9) = -0.6 / 0.1 = -6$
correct v	$\hat{v}_1 = 0.036 / (1 - 0.999) = 0.036 / 0.001 = 36$
denominator	$\sqrt{\hat{v}_1} = 6$
step	$\Delta w = -0.1 \times (-6) / 6 = +0.1$
new parameter	$w_1 = 0.1$

$\hat{m}_1 = g_1$ and $\hat{v}_1 = g_1^2$, exactly

So the corrected first step is $-\eta g_1 / |g_1| = \pm\eta$. **Adam's first step has size exactly η , downhill, whatever the gradient's magnitude.**

Predict first #4

Same step, but delete the two bias-correction lines: use m_1 and v_1 raw.

The corrected first step was $+0.1000$. What is the uncorrected one, and by what factor is it wrong?

Predict first #4

Same step, but delete the two bias-correction lines: use m_1 and v_1 raw.

The corrected first step was $+0.1000$. What is the uncorrected one, and by what factor is it wrong?

```
first gradient          g1          = -6.0000
uncorrected m1 / sqrt(v1)      = -3.1623
corrected  mhat1 / sqrt(vhat1)  = -1.0000
ratio (1-b1)/sqrt(1-b2)       = 3.1623
worst overshoot at t=12, factor 6.57 too big
within 1% of the corrected step only from t=3916
```

Predict first #4

Same step, but delete the two bias-correction lines: use m_1 and v_1 raw.

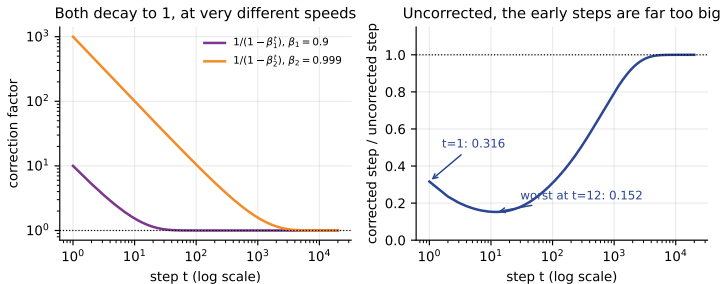
The corrected first step was +0.1000. What is the uncorrected one, and by what factor is it wrong?

```
first gradient          g1          = -6.0000
uncorrected m1 / sqrt(v1)      = -3.1623
corrected  mhat1 / sqrt(vhat1)  = -1.0000
ratio (1-b1)/sqrt(1-b2)       = 3.1623
worst overshoot at t=12, factor 6.57 too big
within 1% of the corrected step only from t=3916
```

Three times too big at $t = 1$, six and a half times at $t = 12$

The two corrections fade at very different rates: β_1 's is within 1% of 1 by step 44, β_2 's only by step **4613**. That gap is the reason bias correction exists.

Bias correction, drawn



Left: the two correction factors $1/(1 - \beta_1^t)$ and $1/(1 - \beta_2^t)$, log-log — one decays in tens of steps, the other in thousands. Right: their combined effect. The uncorrected step is $3.16 \times$ **too big** at $t = 1$, worst at $t = 12$ at $6.57 \times$, and within one per cent only after $t = 3916$.

The two trajectories, run side by side

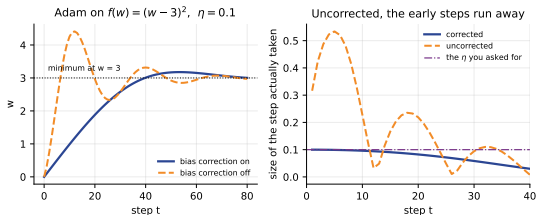
Both runs are Adam on $f(w) = (w - 3)^2$ from $w_0 = 0$ with $\eta = 0.1$. The only difference is the two correction lines.

	t=1	t=2	t=3	t=12
w, corrected	0.1000	0.1999	0.2996	1.1753
w, no bc	0.3162	0.7393	1.2263	4.4123
step size, corrected:	never exceeds 0.1000 (you asked for 0.1)			
step size, no bc:	peaks at 0.5340 = 5.3 times too big			
furthest past the minimum, corrected	3.1773 at t=53			
furthest past the minimum, no bc	4.4123 at t=12			

The two trajectories, run side by side

Both runs are Adam on $f(w) = (w - 3)^2$ from $w_0 = 0$ with $\eta = 0.1$. The only difference is the two correction lines.

	t=1	t=2	t=3	t=12
w, corrected	0.1000	0.1999	0.2996	1.1753
w, no bc	0.3162	0.7393	1.2263	4.4123
step size, corrected:	never exceeds 0.1000 (you asked for 0.1)			
step size, no bc:	peaks at 0.5340 = 5.3 times too big			
furthest past the minimum, corrected	3.1773 at t=53			
furthest past the minimum, no bc	4.4123 at t=12			



By hand: Adam, steps 2 and 3

Carrying $w_1 = 0.1$, $m_1 = -0.6$, $v_1 = 0.036$ forward.

t	g_t	m_t	v_t	$\hat{m}_t$	$\sqrt{\hat{v}_t}$	w_t
1	-6.0000	-0.60000	0.036000	-6.0000	6.0000	0.100000
2	-5.8000	-1.12000	0.069604	-5.8947	5.9008	0.199897
3	-5.6002	-1.56802	0.100897	-5.7861	5.8022	0.299618

Row 2 in full: $m_2 = 0.9(-0.6) + 0.1(-5.8) = -1.12$;
 $\hat{m}_2 = -1.12/(1 - 0.81) = -5.8947$; $\hat{v}_2 = 0.069604/(1 - 0.998001) = 34.819$.

Every step is almost exactly $\eta = 0.1$

0.1000, 0.0999, 0.0997. The ratio $\hat{m}/\sqrt{\hat{v}}$ stays near ± 1 because both moments track the *same* gradient sequence. **Adam moves at a speed you set, not one the gradient sets.**

The same thing in code

```
eta, b1, b2, eps = 0.1, 0.9, 0.999, 1e-8
w, m, v = 0.0, 0.0, 0.0
for t in (1, 2, 3):
    g = 2.0*(w - 3.0)
    m = b1*m + (1 - b1)*g
    v = b2*v + (1 - b2)*g*g
    mh, vh = m/(1 - b1**t), v/(1 - b2**t) # bias correction
    w = w - eta*mh/(np.sqrt(vh) + eps)
```

t	g	m	v	mhat	sqrt(vhat)	w
1	-6.0000	-0.60000	0.036000	-6.0000	6.0000	0.100000
2	-5.8000	-1.12000	0.069604	-5.8947	5.9008	0.199897
3	-5.6002	-1.56802	0.100897	-5.7861	5.8022	0.299618

Six lines. Adam is Adagrad's idea, RMSprop's fix, momentum's direction, and two divisions.

Does bias correction matter in practice?

load_diabetes, $\eta = 0.02$, $B = 32$, with and without the two correction lines:

Adam, eta=0.02, B=32	epoch 1	epoch 5	epoch 30
bias correction on	0.5369	0.4888	0.4929
bias correction off	0.5758	0.4885	0.4960

Does bias correction matter in practice?

load_diabetes, $\eta = 0.02$, $B = 32$, with and without the two correction lines:

Adam, eta=0.02, B=32	epoch 1	epoch 5	epoch 30
bias correction on	0.5369	0.4888	0.4929
bias correction off	0.5758	0.4885	0.4960

The damage is early, then gone

0.5369 against 0.5758 at epoch 1;
indistinguishable by epoch 5. On a
two-minute convex run: invisible.

So why does it exist?

Because on a large model those first
hundred oversized steps land you
somewhere you cannot recover from —
and because η tuned *with* correction is
not η tuned without it.

Does bias correction matter in practice?

`load_diabetes`, $\eta = 0.02$, $B = 32$, with and without the two correction lines:

Adam, eta=0.02, B=32	epoch 1	epoch 5	epoch 30
bias correction on	0.5369	0.4888	0.4929
bias correction off	0.5758	0.4885	0.4960

The damage is early, then gone

0.5369 against 0.5758 at epoch 1;
indistinguishable by epoch 5. On a
two-minute convex run: invisible.

So why does it exist?

Because on a large model those first
hundred oversized steps land you
somewhere you cannot recover from —
and because η tuned *with* correction is
not η tuned without it.

Warmup — ramping η up over the first few thousand steps — is the modern
belt-and-braces version of the same idea.

Six optimisers, one problem

load_diabetes, $B = 32$, 30 epochs, each at its own tuned η :

optimiser	eta	epoch 1	epoch 5	epoch 30
sgd	0.005	0.7393	0.5358	0.4860
momentum	0.005	0.5711	0.4870	0.4846
adagrad	0.050	0.5349	0.4884	0.4859
rmsprop	0.005	0.6613	0.4948	0.4853
adadelta	1.000	0.8157	0.5665	0.4859
adam	0.020	0.5369	0.4888	0.4929
optimum				0.4823

Six optimisers, one problem

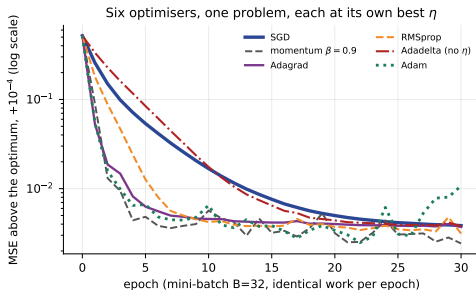
load_diabetes, $B = 32$, 30 epochs, each at its own tuned η :

optimiser	eta	epoch 1	epoch 5	epoch 30
sgd	0.005	0.7393	0.5358	0.4860
momentum	0.005	0.5711	0.4870	0.4846
adagrad	0.050	0.5349	0.4884	0.4859
rmsprop	0.005	0.6613	0.4948	0.4853
adadelat	1.000	0.8157	0.5665	0.4859
adam	0.020	0.5369	0.4888	0.4929
optimum				0.4823

Read the last column before the first

Every one of them lands between 0.4846 and 0.4929. On a well-conditioned convex problem with a tuned η , **the choice of optimiser is worth about one per cent** — and Adam is the *worst* at epoch 30, because it never settles.

The same six, drawn



Distance above the optimum, log scale. The adaptive methods win the first two epochs and then flatten into a noise band; momentum quietly keeps descending. **If you read only the first epoch you would draw the opposite conclusion from the right one.**

And the sparse problem, all five

The same forty-feature problem from the start of the lecture, ten epochs:

optimiser	eta	MSE	dense	5 rarest
sgd	0.010	0.0853	1.0004	0.1562
momentum	0.001	0.0853	0.9998	0.1557
adagrad	0.100	0.0105	0.9999	0.8631
rmsprop	0.005	0.0106	1.0003	0.9846
adam	0.050	0.0125	0.9990	1.0051

And the sparse problem, all five

The same forty-feature problem from the start of the lecture, ten epochs:

optimiser	eta	MSE	dense	5 rarest
sgd	0.010	0.0853	1.0004	0.1562
momentum	0.001	0.0853	0.9998	0.1557
adagrad	0.100	0.0105	0.9999	0.8631
rmsprop	0.005	0.0106	1.0003	0.9846
adam	0.050	0.0125	0.9990	1.0051

Here the choice of optimiser is worth a factor of eight

Momentum does not help at all — it averages the direction, not the scale, so a rare feature's rare gradient stays rare. Every *per-coordinate* method fixes it. **Sparse data is where adaptive methods earn their keep.**

Predict first, one last time

A grid of 25 learning rates from 10^{-4} to 3. For each optimiser, count how many of them land within 5% of the optimum in 10 epochs.

Which optimiser tolerates the widest range of η — and how many decades wide is that range?

Predict first, one last time

A grid of 25 learning rates from 10^{-4} to 3. For each optimiser, count how many of them land within 5% of the optimum in 10 epochs.

Which optimiser tolerates the widest range of η — and how many decades wide is that range?

sgd	8/25 grid points work:	0.0048 to 0.0965	(1.31 decades)
momentum	10/25 grid points work:	0.0006 to 0.0266	(1.68 decades)
adagrad	13/25 grid points work:	0.0173 to 3.0000	(2.24 decades)
rmsprop	9/25 grid points work:	0.0020 to 0.0628	(1.49 decades)
adam	9/25 grid points work:	0.0031 to 0.0965	(1.49 decades)

Predict first, one last time

A grid of 25 learning rates from 10^{-4} to 3. For each optimiser, count how many of them land within 5% of the optimum in 10 epochs.

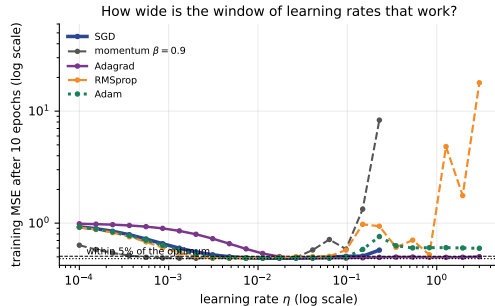
Which optimiser tolerates the widest range of η — and how many decades wide is that range?

sgd	8/25 grid points work:	0.0048 to 0.0965	(1.31 decades)
momentum	10/25 grid points work:	0.0006 to 0.0266	(1.68 decades)
adagrad	13/25 grid points work:	0.0173 to 3.0000	(2.24 decades)
rmsprop	9/25 grid points work:	0.0020 to 0.0628	(1.49 decades)
adam	9/25 grid points work:	0.0031 to 0.0965	(1.49 decades)

Adagrad — still working at the top of the grid

Its own $1/\sqrt{t}$ decay rescues an η that would destroy every other method; 3.0 is where we stopped looking, not where it failed. **This, not final accuracy, is what adaptive optimisers actually buy: a wider target to aim at.**

The window of usable learning rates, drawn



Every curve is a bathtub with a hard right-hand wall: below the wall a run is merely slow, above it the run is destroyed. Adagrad's bathtub is 2.24 decades wide, plain SGD's is 1.31.

Back to the ravine

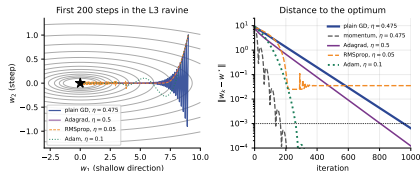
$f(w) = 0.01w_1^2 + 2w_2^2$, $\kappa = 200$, start at (9, 1):

optimiser	eta	iters to $ w < 1e-3$	$ w $ after 1200
gd	0.475	954	9.99e-04
momentum	0.475	135	7.88e-04
adagrad	0.500	805	9.97e-04
rmsprop	0.050	never	3.54e-02
adam	0.100	262	9.89e-04

Back to the ravine

$f(w) = 0.01w_1^2 + 2w_2^2$, $\kappa = 200$, start at (9, 1):

optimiser	eta	iters to $\ w\ < 1e-3$	$\ w\ $ after 1200
gd	0.475	954	9.99e-04
momentum	0.475	135	7.88e-04
adagrad	0.500	805	9.97e-04
rmsprop	0.050	never	3.54e-02
adam	0.100	262	9.89e-04



Momentum still wins here

A ravine is a *curvature* problem, and adaptive methods measure *gradients*. Momentum's $\sqrt{\kappa}$ acceleration is the right tool: 135 beats everything.

RMSprop never arrives

Constant η , bounded denominator: a limit cycle at 3.5×10^{-2} .

Two numerical details that will cost you a run

```
RMSprop, epsilon = 1e-08 MSE after 10 epochs 0.4864
RMSprop, epsilon = 1e-06 MSE after 10 epochs 0.4864
RMSprop, epsilon = 1e-04 MSE after 10 epochs 0.4864
RMSprop, epsilon = 1e-02 MSE after 10 epochs 0.4864
RMSprop, epsilon = 1e-01 MSE after 10 epochs 0.4870
RMSprop, epsilon = 1e+00 MSE after 10 epochs 0.5185
```

Two numerical details that will cost you a run

```
RMSprop, epsilon = 1e-08 MSE after 10 epochs 0.4864
RMSprop, epsilon = 1e-06 MSE after 10 epochs 0.4864
RMSprop, epsilon = 1e-04 MSE after 10 epochs 0.4864
RMSprop, epsilon = 1e-02 MSE after 10 epochs 0.4864
RMSprop, epsilon = 1e-01 MSE after 10 epochs 0.4870
RMSprop, epsilon = 1e+00 MSE after 10 epochs 0.5185
```

```
Adam, decay off epoch 5 MSE 0.4888 epoch 30 MSE 0.4929
Adam, decay on epoch 5 MSE 0.4877 epoch 30 MSE 0.4850
```

Two numerical details that will cost you a run

```
RMSprop, epsilon = 1e-08 MSE after 10 epochs 0.4864
RMSprop, epsilon = 1e-06 MSE after 10 epochs 0.4864
RMSprop, epsilon = 1e-04 MSE after 10 epochs 0.4864
RMSprop, epsilon = 1e-02 MSE after 10 epochs 0.4864
RMSprop, epsilon = 1e-01 MSE after 10 epochs 0.4870
RMSprop, epsilon = 1e+00 MSE after 10 epochs 0.5185
```

```
Adam, decay off epoch 5 MSE 0.4888 epoch 30 MSE 0.4929
Adam, decay on epoch 5 MSE 0.4877 epoch 30 MSE 0.4850
```

ϵ **is not a hyperparameter until it is.** Anything below 10^{-2} is identical here; at $\epsilon = 1$ it dominates $\sqrt{s}$ and RMSprop degenerates into plain SGD.

Adam got worse between epoch 5 and epoch 30 ($0.4888 \rightarrow 0.4929$). A $1/(1+ct)$ decay fixes it: 0.4850. Adaptive does not mean schedule-free.

Which one should you use?

	Reach for it when	Watch out for
SGD + momentum	you can afford to tune and you want the best final number	narrowest usable η window (1.31 decades)
Adagrad	the features are sparse ; short runs	the step decays as $1/\sqrt{t}$ and stalls
RMSprop	non-stationary objectives; recurrent nets	no convergence guarantee; hovers
Adadelta	you have no time to tune anything	slowest start of the six
Adam	the default: anything new, first attempt	does not settle; decay η at the end

The honest recipe

Standardise. Start with Adam at $\eta = 10^{-3}$ and $B = 32$. If the result matters, tune SGD + momentum against it — on our data it won, 0.4846 against 0.4929.

Summary — and Unit III in twelve lines

1. One η cannot serve two coordinates whose curvatures differ by κ . In our ravine the ideal steps were 50 and 0.25.
2. Every optimiser today is $w_i \leftarrow w_i - (\eta/\sqrt{s_i}) g_i$. They differ only in what s_i is.
3. **Adagrad**: $s_t = s_{t-1} + g_t^2$. It turned one η into forty, spread $39\times$, and lifted the rare-feature weights from 0.156 to 0.863.
4. Adagrad's sum grows linearly forever: measured, the effective step fell $9.4\times$ over 2800 updates and keeps falling as $1/\sqrt{t}$.
5. Chasing a distant minimum it needed **5,092** updates to reach what RMSprop reached in **100**.
6. **RMSprop**: replace the sum by $\gamma s + (1 - \gamma)g^2$. Same bowl, same η : Adagrad reached 1.064 in three steps, RMSprop 2.650.
7. **Adadelta**: the units of $\eta g/\sqrt{s}$ are wrong; supply $\text{RMS}[\Delta w]_{t-1}$ instead and η disappears. Multiply the loss by 100 and its answer moves by 0.02%; SGD's diverges.
8. **Adam** = momentum + RMSprop + bias correction. By hand: $w = 0.1000, 0.1999, 0.2996$ — each step almost exactly η .
9. Without bias correction the same run took steps of 0.534 instead of 0.100 and shot to 4.41 past a minimum at 3.
10. With a tuned η all six landed within one per cent of each other. **Adaptive methods buy tuning robustness, not accuracy**: Adagrad worked over 2.24 decades of η , plain SGD over 1.31.
11. In a deterministic ravine *momentum still wins* (135 against Adam's 262): adaptive methods see gradients, not curvature. On sparse data the ordering reverses completely.
12. Adam drifted from 0.4888 to 0.4929 between epochs 5 and 30. **Decay the learning rate at the end anyway.**

Before the next lecture

Read

notes.pdf — the units derivation for Adadelta, the full bias-correction algebra, and **10 practice questions with worked solutions**.

Do

Questions 3, 5 and 9. Question 5 is Adam by hand from a fresh start; question 9 is a debugging story you will meet for real.

Unit III is now complete

How to descend, then how fast, per coordinate.

Everything after this assumes you can write any of these five updates from memory.

Next: Unit IV — data cleaning

And a warning: no optimiser on these slides can rescue an unstandardised feature matrix. We measured what that costs.

Materials: <https://github.com/tali7c/Applied-Machine-Learning>

Sources: Zhang, Lipton, Li and Smola, *Dive into Deep Learning*, Ch. 12 (Optimization Algorithms); Deisenroth, Faisal and Ong, *Mathematics for Machine Learning*, Ch. 7.; Kingma and Ba, "Adam: A Method for Stochastic Optimization," ICLR 2015.; Ruder, "An overview of gradient descent optimization algorithms," arXiv:1609.04747, 2016.